進階程式設計
Advanced Program Design

L6

Chia-Chi Chang
ccchang@cc.ncue.edu.tw

浮點數陷阱

- 二進位浮點數算術標準(IEEE 754)主要是讓微處理器系統採用符號數值表示法去表示二進位格式的浮點數。
  - https://www.h-schmidt.net/FloatConverter/IEEE754.html
- 在電腦科學裡使用的浮點數，已經由電氣電子工程師協會IEEE制定標準稱為IEEE 754

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
int main(int argc, char *argv[]) {
    int loop;
    float i = 0.7, j = 0.4;
    double k = 0.7, p = 0.4;


    printf("i = %f, j = %f\n",i,j);
    printf("i = %.20f, j = %.20f\n",i,j);
    printf("================================\n");
    printf("k = %lf, p = %lf\n",k,p);
    printf("k = %.20lf, p = %.20lf\n",k,p);
    printf("================================\n");
    system("PAUSE");
    return 0;
}
```

```
i = 0.700000, j = 0.400000
i = 0.69999998807907104000, j = 0.40000000596046448000
================================
k = 0.700000, p = 0.400000
k = 0.69999999999999996000, p = 0.40000000000000002000
================================
Press any key to continue . . .
```

浮點數的比較

- 無論是單精度浮點數或倍精度浮點數，貿然進行==或!=會出現潛在錯誤。

```
01 int main(int argc, char *argv[]) {
02     float float_i = 0.1, float_j = 0.2;
03     double double_i = 0.1, double_j = 0.2;
04
05     if((float_i+float_j) == 0.3) {
06         printf("float_i == float_j\n");
07     } else {
08         printf("float_i != float_j\n");
09     }
10     if((double_i+double_j) == 0.3) {
11         printf("double_i == double_j\n");
12     } else {
13         printf("double_i != double_j\n");
14     }
15     return 0;
16 }
```

```
float_i != float_j
double_i != double_j
```

浮點數的比較

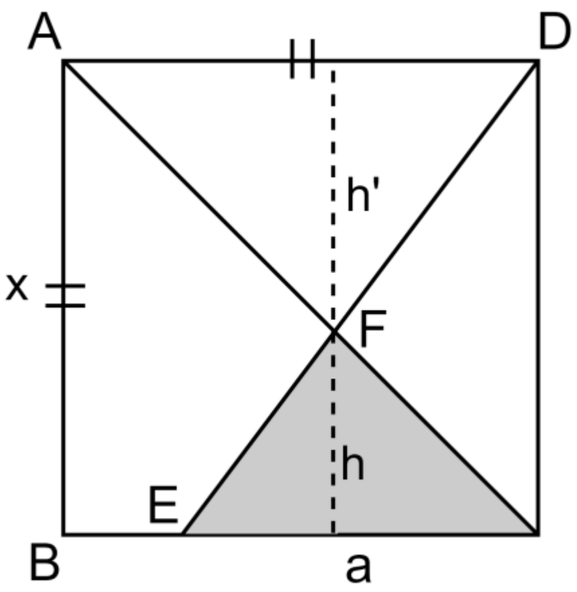
- 利用float.h標頭檔定義FLT_EPSILON和DBL_EPSILON，去判斷兩個浮點數值是否在EPSILON誤差
  - 單精度浮點數誤差：FLT_EPSILON(IEC 60559) = 0.00000011920928955078
  - 倍精度浮點數誤差：DBL_EPSILON(IEC 60559) = 0.00000000000000022204
- 自行設計的EPSILON，如#define MY_EPSILON 1e-8
- 結合絕對值fabsf(用於單精度浮點數)或fabs(用於倍精度浮點數)，參考如下程式碼所述，則可認定兩個浮點數值是相同。

```c
#include <stdio.h>
#include <stdlib.h>
#include <float.h>

int main(int argc, char *argv[]) {
    printf("FLT_EPSILON(IEC 60559) = %.20lf\n",FLT_EPSILON);
    printf("DBL_EPSILON(IEC 60559) = %.20lf\n",DBL_EPSILON);
    return 0;
}
```

```
FLT_EPSILON(IEC 60559) = 0.00000011920928955078
DBL_EPSILON(IEC 60559) = 0.00000000000000022204
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <float.h>
int main(int argc, char *argv[]) {
    float float_i = 0.1, float_j = 0.2;
    double double_i = 0.1, double_j = 0.2;

    if(fabsf((float_i+float_j) - 0.3) < FLT_EPSILON) {
        printf("float_i == float_j\n");
    } else {
        printf("float_i != float_j\n");
    }
    if(fabs((double_i+double_j) - 0.3) < DBL_EPSILON) {
        printf("double_i == double_j\n");
    } else {
        printf("double_i != double_j\n");
    }
    return 0;
}
```

Q8



$$\frac{h}{h'}=\frac{a}{x}\Rightarrow hx=a\cdot h'=a(x-h)\Rightarrow h=\frac{xa}{x+a}$$

$$\text{Shadow Area: }\frac{a\cdot h}{2}=\frac{a}{2}\cdot\frac{xa}{x+a}=\frac{x\cdot a^2}{2(x+a)}$$

- 第5行陰影面積算式和第10行陰影面積算式是一致，但是浮點式的誤差造成答案會是131.63(正確答案)和131.62。
- 13162.5與13162.499999999998，浮點數誤差會讓評測系統判定錯誤

```
01 int main(int argc, char *argv[]) {
02     int area_A = 6084, a =18;
03     double shadowArea, num;
04
05     //ShadowArea * 100 = (a * a * x / 2 * (x + a)) * 100 => 50 * a^2 * x^2 / (x^2 + a * x)
06     shadowArea = 50 * a * a * area_A / (area_A + a * sqrt(area_A));
07     printf("ShadowArea(1) = %.15lf\n",shadowArea);
08     printf("round(ShadowArea(1)) = %.3lf\n",round(shadowArea)/100);
09     printf("================================\n");
10     //ShadowArea * 100 = (50 * a) / (((area_A + a * sqrt(area_A)) / (area_A * a))
11     int eq1 = 50 * a;
12     double eq2 = (area_A + a * sqrt(area_A)) / (area_A * a);
13     printf("ShadowArea(2) = %.15lf\n",eq1/eq2);
14     printf("round(ShadowArea(2)) = %.3lf\n",round(eq1/eq2)/100);
15     return 0;
16 }
```

```
ShadowArea(1) = 13162.500000000000000
round(ShadowArea(1)) = 131.630
================================
ShadowArea(2) = 13162.499999999998000
round(ShadowArea(2)) = 131.620
```

```
double calAreaofShadow(int area_A, int a) {
    int eq1 = 50 * a;
    double eq2 = (area_A + a * sqrt(area_A)) / (area_A * a);
    double areaShadow = eq1 / eq2;

    if(fabs((floor(areaShadow)+0.5)-areaShadow) < 1e-8) {
        areaShadow = floor(areaShadow) + 0.5;
    }
    return areaShadow;
}
```

如何縮短誤差？

- 利用二進位格式去表示浮點數，本身就會有誤差值存在。
- 不同的編譯器產生的浮點數也存在差異性。

改善的方法：

1. 避免兩個差距甚大的浮點數值進行運算。
2. 多利用整數進行前期運算，最後才進行浮點運算。
3. 利用float.h裡定義的FLT_EPSILON或DBL_EPSILON或自製的EPSILON(如1e-8)，去判斷計算的結果與理想值的差距是否小於EPSILON值，若小於EPSILON值就把計算的結果修正成理想值，藉以消除誤差。

C的後綴字(suffix)

- C提供整數和浮點數的後綴字(接尾詞, suffix)，來明確指出整數和浮點數的資料型別
- 整數的後綴字：u/U代表unsigned, l或L代表long int, ll或LL代表long long int
  - ul的組合代表unsigned long int, llu的組合代表unsigned long long int
    - 79 //(預設int，再來long int，然後unsigned long int)
    - 174u //unsigned int
    - 423ul //unsigned long int
    - 5llu //unsigned long long int
- 浮點數的後綴字：f/F代表float, l/L代表long double
  - 4.1 //double
  - 1.28f //float
  - 3.14159L //long double

後綴字影響

```c
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    unsigned long long i = 65536 * 70000;
    unsigned long long j = 4587520000;

    if(i == j) {
        printf("i == j\n");
    } else {
        printf("i != j\n");
    }
    printf("i = %llu\n",i);
    printf("j = %llu\n",j);
    return 0;
}
```

```
i != j
i = 292552704
j = 4587520000
```

```c
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    unsigned long long i = 65536llu * 70000;
    unsigned long long j = 4587520000;

    if(i == j) {
        printf("i == j\n");
    } else {
        printf("i != j\n");
    }
    printf("i = %llu\n",i);
    printf("j = %llu\n",j);
    return 0;
}
```

```
i == j
i = 4587520000
j = 4587520000
```

後綴字/強制轉換

```c
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    double i = 1 / 3;

    printf("i = %.2lf\n",i);
    return 0;
}
```

```
i = 0.00
```

```c
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    double i = 1.0 / 3;
    double j = (float)1 / 3;

    printf("i = %.2lf\n",i);
    printf("j = %.2lf\n",j);
    return 0;
}
```

```
i = 0.33
j = 0.33
```

列印long double内容

```c
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    unsigned long long int i = 65536llu * 70000;
    long double j = i;

    printf("i = %llu\n",i);
    printf("j = %.2Lf\n",j);
    return 0;
}
```

```
i = 4587520000
j = 0.00
```

```c
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    unsigned long long int i = 65536llu * 70000;
    long double j = i;

    printf("i = %llu\n",i);
    __mingw_printf("j = %.2Lf\n",j);
    return 0;
}
```

```
i = 4587520000
j = 4587520000.00
```

The End